

Allen–Cahn: background, discretisation, and experiments

August 14, 2026

A self-contained account of the Allen–Cahn part of the project: the energy, the gradient flow, the discretisation as it is actually implemented, the two time-stepping schemes and the two physics losses they induce, the autodiff-through-time question, and a summary of every experiment we have run. Everything is checked against the source; pointers are `path:line` at the time of writing. The companion document `notes/ac_autoregressive/ac_autoregressive.tex` carries the longer derivations (numerical regime analysis, conditioning, mesh-consistent scaling); this document is the summary and the experimental record.

1 The energy

Allen–Cahn is the L^2 gradient flow of the Ginzburg–Landau free energy. On $\Omega = (0, 1)^2$ with homogeneous Dirichlet conditions,

$$E(u) = \frac{a^2}{2} \int_{\Omega} |\nabla u|^2 dx + \epsilon^2 \int_{\Omega} W(u) dx, \quad W(s) = \frac{1}{4} (s^2 - 1)^2. \quad (1)$$

The first term penalises interfaces, the double well W has minima at $s = \pm 1$ and forces the field towards the two pure phases. The competition sets an interface width $\ell \sim a/\epsilon$.

Two conventions that are easy to get wrong and are fixed throughout the code and this project:

- ϵ multiplies the *reaction*, not the Laplacian, and it enters *squared*. Large ϵ means a stiff, sharply-separating problem. This follows TensorMesh’s `examples/diffusion/allen-cahn` and is the opposite of the more common $\epsilon \rightarrow 0$ sharp-interface convention.
- The diffusion coefficient is a^2 ; we use $a = 1$ in every experiment, so ϵ alone controls stiffness.

2 The gradient flow

The L^2 gradient flow $\partial_t u = -\delta E/\delta u$ of (1) is

$$\partial_t u = a^2 \Delta u + \epsilon^2 u(1 - u^2), \quad u|_{\partial\Omega} = 0, \quad (2)$$

since $-\delta E/\delta u = a^2 \Delta u - \epsilon^2 W'(u)$ and $W'(s) = s^3 - s$. The structural consequence we lean on repeatedly is that E is a Lyapunov functional,

$$\frac{d}{dt} E(u(t)) = -\|\partial_t u(t)\|_{L^2}^2 \leq 0, \quad (3)$$

so **the energy must be non-increasing along any correct trajectory**. We use this as a diagnostic for learned rollouts: an energy curve that climbs is a blow-up, regardless of what the error metric says (`ACProblem.energy`, `tensorpils/physics.py:282`).

3 Discretisation: nodal interpolation of the nonlinearity

Space is discretised with bilinear Q_1 elements on a structured quadrilateral grid of the unit square (`structured_quad_mesh`, `tensorpils/meshing.py:24`); the stiffness A and mass M are assembled once by TensorMesh with Gauss quadrature of order 4 per axis (`physics.py:64--65`). This is the same A , M , and Dirichlet masking as the Poisson setting: the boundary condition is imposed by projection, not condensation.

The nonlinearity is treated by **nodal (group / product) interpolation**: rather than integrating $W'(u_h)$ exactly, we interpolate the nonlinear function of the nodal values into the same Q_1 space,

$$\int_{\Omega} W'(u_h) \phi_i dx \approx \int_{\Omega} \left(\sum_j W'(u_j) \phi_j \right) \phi_i dx = [M W'(\mathbf{u})]_i, \quad (4)$$

with $W'(\mathbf{u})$ applied entrywise. So the cubic reaction becomes $M(\mathbf{u} - \mathbf{u}^3)$ and never requires quadrature of a cubic integrand. The same convention gives the discrete energy

$$E_h(\mathbf{u}) = \frac{1}{2} a^2 \mathbf{u}^\top A \mathbf{u} + \epsilon^2 \mathbf{1}^\top M W(\mathbf{u}), \quad (5)$$

i.e. the double well is evaluated nodally and then integrated exactly against the Q_1 basis (`physics.py:282--298`). Nodal interpolation is what makes the reaction Jacobian a *diagonal scaling of M* , which is exactly what the Newton solve and the preconditioner below exploit.

4 Time stepping I: minimizing movement

Continuous (JKO / De Giorgi). The gradient-flow structure gives a variational time discretisation: given u^n , define

$$u^{n+1} = \arg \min_u \left\{ E(u) + \frac{1}{2\Delta t} \|u - u^n\|_{L^2}^2 \right\}. \quad (6)$$

The optimality condition is $\delta E / \delta u + (u - u^n) / \Delta t = 0$, i.e. exactly backward Euler applied to (2). Choosing u^{n+1} as the minimiser rather than as the root of a residual guarantees $E(u^{n+1}) + \frac{1}{2\Delta t} \|u^{n+1} - u^n\|^2 \leq E(u^n)$ and hence unconditional energy decay. The catch is that E is *not* convex — the concave part $-\frac{\epsilon^2}{2} \int u^2$ of the double well spoils it — so (6) is not a convex problem, and for Δt large relative to $1/\epsilon^2$ the objective can lose its unique minimiser. This is what the convex–concave split of §5 repairs.

Discrete (what we implement). We use the *convex–concave* minimizing-movement objective throughout, so §5 supplies the split and this section supplies the variational form. Splitting $E = E_{\text{cvx}} + E_{\text{ccv}}$ with

$$E_{\text{cvx}}(u) = \frac{a^2}{2} \int |\nabla u|^2 + \frac{\epsilon^2}{4} \int u^4 + \frac{\epsilon^2}{4} |\Omega|, \quad E_{\text{ccv}}(u) = -\frac{\epsilon^2}{2} \int u^2, \quad (7)$$

(so that $E_{\text{cvx}} + E_{\text{ccv}} = E$, using $W(s) = \frac{1}{4}(s^4 - 2s^2 + 1)$), and linearising only the concave part about u^n , the step is the unique minimiser of the *convex* objective

$$J(u) = E_{\text{cvx}}(u) + \langle DE_{\text{ccv}}(u^n), u \rangle + \frac{1}{2\Delta t} \|u - u^n\|_{L^2}^2, \quad DE_{\text{ccv}}(u^n) = -\epsilon^2 u^n. \quad (8)$$

In FEM variables, with the quartic evaluated nodally against the *lumped* mass $M\mathbf{1}$ (`ACProblem.mm_objective, physics.py:240--280`),

$$J(\mathbf{u}) = \frac{a^2}{2} \mathbf{u}^\top A \mathbf{u} + \frac{\epsilon^2}{4} \left[(\mathbf{u}^4)^\top M \mathbf{1} + |\Omega| \right] - \epsilon^2 (\mathbf{u}^n)^\top M \mathbf{u} + \frac{1}{2\Delta t} (\mathbf{u} - \mathbf{u}^n)^\top M (\mathbf{u} - \mathbf{u}^n), \quad (9)$$

whose gradient is $\nabla J = M \frac{\mathbf{u} - \mathbf{u}^n}{\Delta t} + a^2 A \mathbf{u} + \epsilon^2 \text{diag}(M \mathbf{1}) \mathbf{u}^3 - \epsilon^2 M \mathbf{u}^n$. Because J is convex with Hessian $\nabla^2 J = M/\Delta t + a^2 A + 3\epsilon^2 \text{diag}(M \mathbf{1}) \text{diag}(\mathbf{u}^2)$, which is SPD for every $\mathbf{u}$ and every Δt , this is the **Deep-Ritz analogue for time stepping**: the loss is an energy to be minimised, not a residual to be squared, and it inherits $\kappa(\nabla^2 J)$ rather than κ^2 . That is the entire reason it behaves differently from the least-squares loss.

Small inconsistency worth knowing. J uses the *lumped* mass on the quartic $((\mathbf{u}^4)^\top M \mathbf{1})$ while the diagnostic energy (5) uses the *consistent* mass $(\mathbf{1}^\top M W(\mathbf{u}))$. Both are legitimate nodal-interpolation choices and they agree to quadrature order, but they are not literally the same functional, so J and E_h should not be expected to match to machine precision.

5 Time stepping II: convex–concave (Eyre) splitting

Continuous. The Eyre split treats the convex part implicitly and the concave part explicitly:

$$\frac{u^{n+1} - u^n}{\Delta t} = a^2 \Delta u^{n+1} - \epsilon^2 (u^{n+1})^3 + \epsilon^2 u^n. \quad (10)$$

Compare fully implicit backward Euler, which puts the whole reaction $\epsilon^2(u^{n+1} - (u^{n+1})^3)$ at the new level. The two differ *only in the linear reaction term*: implicit at u^{n+1} (backward Euler) versus explicit at u^n (convex–concave). The split is unconditionally energy-stable and is the standard scheme for Allen–Cahn at stiff ϵ ; it is exactly the optimality condition of (8), so *the convex–concave step and the minimizing-movement step are the same step*, once via a residual and once via an objective.

Discrete. With nodal interpolation the two residuals are (`ACProblem.residual, physics.py:205--237`), both scaled by $1/\Delta t$:

$$R_{\text{BE}}(\mathbf{u}^n, \mathbf{u}^{n+1}) = M \frac{\mathbf{u}^{n+1} - \mathbf{u}^n}{\Delta t} + a^2 A \mathbf{u}^{n+1} - \epsilon^2 M (\mathbf{u}^{n+1} - (\mathbf{u}^{n+1})^3), \quad (11)$$

$$R_{\text{CC}}(\mathbf{u}^n, \mathbf{u}^{n+1}) = M \frac{\mathbf{u}^{n+1} - \mathbf{u}^n}{\Delta t} + a^2 A \mathbf{u}^{n+1} + \epsilon^2 M ((\mathbf{u}^{n+1})^3 - \mathbf{u}^n). \quad (12)$$

The $1/\Delta t$ scaling is deliberate: it makes a least-squares loss built on either residual have a Δt -independent gradient scale. The corresponding Newton Jacobians are

$$J_{\text{CC}} = \frac{M}{\Delta t} + a^2 A + 3\epsilon^2 M \text{diag}(\mathbf{u}^2), \quad J_{\text{BE}} = \frac{M}{\Delta t} + a^2 A + \epsilon^2 M \text{diag}(3\mathbf{u}^2 - 1). \quad (13)$$

J_{CC} is SPD for every $\mathbf{u}$ and Δt (the reaction term is a non-negative diagonal scaling of M); J_{BE} is not, which is why the reference solver defaults to convex–concave.

The reference trajectory. Allen–Cahn has no analytical solution, so the ground truth is a FEM solve: each step drives $R = 0$ by Newton on the interior degrees of freedom, with a **batched sparse** linear solve and **always in float64** regardless of the caller’s dtype — `fp32` cannot reach `newton_tol = 10-8` and leaves Newton spinning to `newton_max` every step (`ACProblem.fem_reference`, `physics.py:380--458`). The Jacobian is reassembled per Newton step as a value update on the shared sparsity pattern of $L = M/\Delta t + a^2 A$, since the reaction contributes only a column scaling of M — again a consequence of nodal interpolation.

Two things follow, and both matter for reading the results:

- The learned stepper is measured against the *discrete* convex–concave trajectory, not against physical Allen–Cahn. This is well-posed at every ϵ (the CC scheme is unconditionally stable), but at large ϵ the reference is under-resolved and is no longer physical Allen–Cahn.
- Data generation and the physics loss use the same integrator by default, so the reference *exactly* zeroes the residual the loss penalises (`ACTrainer`, `trainer.py:812`). Decoupling them (`--ac_loss_integrator`) is an intentional experiment, not an accident.

6 The two physics losses

Both losses act on an autoregressive rollout $\mathbf{u}^0, \dots, \mathbf{u}^{L-1}$ where $\mathbf{u}^0$ is the ground-truth seed frame and the rest are model predictions, with a discount γ^k over rollout steps and a normalisation by $\sum_k \gamma^k$:

$$\mathcal{L}_{\text{LS}} = \frac{1}{\sum_k \gamma^k} \sum_k \gamma^k \frac{1}{2} \|P R(\mathbf{u}^k, \mathbf{u}^{k+1})\|_2^2 \quad (\text{ACGalerkinLoss, losses.py:255}) \quad (14)$$

$$\mathcal{L}_{\text{MM}} = \frac{1}{\sum_k \gamma^k} \sum_k \gamma^k J(\mathbf{u}^k; \mathbf{u}^{k+1}) \quad (\text{ACMinMovementLoss, losses.py:310}) \quad (15)$$

with $P = I$ unless preconditioning is requested. The squared residual is **summed over nodes and averaged over the batch** (keeping the gradient $O(1)$ rather than $O(1/N)$); J is already a spatial functional, so no per-node reduction is applied. Note that $\mathcal{L}_{\text{MM}}$ is an energy and is routinely *negative* — in the reference run below it converges to ≈ -120 — so its numerical value is not comparable across losses and only the validation error is meaningful for model selection.

Preconditioning the least-squares loss. $\mathcal{L}_{\text{LS}}$ has Gauss–Newton Hessian $\approx J^\top J$, so it inherits $\kappa(J)^2$ — the same squaring that kills the Poisson least-squares loss. We precondition with a **frozen-coefficient** approximation of the Jacobian: setting $\mathbf{u}^2 \equiv 1$ (exact in the bulk of either phase, wrong only inside the interface) in (13) gives the constant, SPD, u -independent operator

$$J_0 = a^2 A + c M, \quad c = \frac{1}{\Delta t} + 3\epsilon^2, \quad (16)$$

a screened-Poisson / reaction–diffusion operator. $P \approx J_0^{-1}$ is realised by the *same* geometric-multigrid V-cycle as in the Poisson experiments, simply built on $a^2 A + c M$ instead of A (`GeometricMultigrid` with `mg_a2/mg_c`, `preconditioners/multigrid.py:111--125`; wired at `trainer.py:825--838`). The mass shift only makes the operator more diagonally dominant, so weighted-Jacobi smoothing is at least as effective as in the pure Poisson case. Preconditioning applies to the least-squares form only — it is meaningless for J , whose Hessian is already the un-squared $\nabla^2 J$.

7 Autodiff through time

The rollout has *two independent detach knobs*, which is the part that is easy to conflate (RolloutTrainer._rollout, trainer.py:622--640; resolution at trainer.py:820):

1. **Rollout-input detach.** Whether step $k+1$ receives $\mathbf{u}^k$ or $\mathbf{u}^k.\text{detach}()$ as *network input*. Detaching makes every stored frame a one-step function of its input — no backpropagation through time at all. This is the *pushforward trick* (Brandstetter, Worrall & Welling, 2022).
2. **Loss-coupling detach.** Whether the *previous frame* is differentiated *inside* the loss — the first argument of $R(\mathbf{u}^k, \mathbf{u}^{k+1})$, or the proximal centre $\mathbf{u}^k$ in J .

The three exposed modes are

--bptt_mode	input detach	coupling detach	gradient
full_bptt	no	no	backprop through the entire rollout
detach_prev	no	yes	full BPTT, previous frame frozen in the loss
pushforward	yes	yes	one-step

The data loss has no coupling term, so `detach_prev` $\equiv$ `full_bptt` there.

Why the coupling detach is not cosmetic. For the least-squares residual, $\frac{1}{2}\|R\|^2$ is *self-balancing*: its gradient vanishes in *both* arguments at $R = 0$, so leaving the coupling attached is harmless, and the class default is not to detach. For minimizing movement it is different: $\nabla_{\mathbf{u}^k} J$ does *not* vanish at the correct step — J is minimised over its second argument only, and the proximal centre is data. Backpropagating through it therefore rewards moving the *centre*, distorting the trajectory. The MM class default is to detach. §8.3 shows this is worth roughly two orders of magnitude, and it is the single clearest experimental result we have about autodiff through time.

8 Experiments

All runs: FNO (16×16 modes, width 64, 5 layers), Adam with cosine $10^{-3} \rightarrow 10^{-4}$, batch 32, $n_{\text{train}}/n_{\text{val}}/n_{\text{test}} = 1024/128/256$, `seed=42`, $a = 1$, zero Dirichlet, convex–concave reference. Initial conditions are multi-frequency, $u_0 = \frac{\pi}{K^2} \sum_{i,j=1}^K a_{ij} (i^2 + j^2)^{-1/2} \sin(\pi i x) \sin(\pi j y)$ with $a_{ij} \sim \text{Unif}[-1, 1]$ (amplitude ≈ 0.15), so K controls how rich the initial data is. Metric is the FEM (mass-matrix) relative L^2 against the reference; “space-time” averages over the rollout.

8.1 ac_loss_comparison — does the loss matter at all?

64^2 , $\Delta t = 0.0025$, $\epsilon = 2$, $K = 4$, 10-step rollout, 1000 epochs. Four arms: data-driven, backward-Euler least squares, convex–concave least squares, minimizing movement. Arm 2 trains the *BE* residual against *CC* data on purpose, to test whether the physics-loss integrator matters. **Outcome:** at $\epsilon = 2$ the dynamics are diffusion-dominated (the double well barely engages) and all four losses land close together — an informative null. This motivated the stress test.

8.2 ac_stresstest — sweep the reaction strength

Same setup, $\epsilon \in \{2, 4, 8, 12, 16, 20, 24, 32\}$ (24 runs), 500 epochs, three losses. Best space-time relative L^2 :

ϵ	data-driven	least squares	min. movement
2	0.024	0.022	0.013
4	0.025	0.004	0.005
8	0.011	0.004	0.004
12	0.011	0.004	0.011
16	0.999	0.975	0.021
20	1.000	0.991	0.056
24	1.000	0.995	0.095
32	0.981	0.998	1.011

Read at face value this is the headline Allen–Cahn story: past $\epsilon \approx 12$ both data-driven and bare least squares collapse to $\approx 100\%$ error while minimizing movement degrades gracefully until it too fails at $\epsilon = 32$. **But see §9 before using these numbers.**

8.3 `compare_bptt` — autodiff modes $\times$ reaction strength

64^2 , $\Delta t = 0.0025$, $K = 4$, 500 epochs, $\epsilon \in \{4, \dots, 32\}$, three losses $\times$ their modes = 64 runs. Best space-time relative L^2 :

loss	mode	4	8	12	16	20	24	28	32
data	<code>full</code>	0.010	0.011	0.007	0.009	0.010	0.012	0.061	0.038
data	<code>push</code>	0.009	0.007	0.006	0.005	0.006	0.009	0.010	0.017
LS	<code>full</code>	0.023	0.004	0.004	0.006	0.012	0.995	0.997	0.998
LS	<code>detach</code>	0.004	0.004	0.006	0.008	0.015	0.028	0.044	1.000
LS	<code>push</code>	0.004	0.004	0.005	0.012	0.015	0.039	0.034	0.035
MM	<code>full</code>	0.408	0.964	1.018	2.854	2.394	2.091	1.736	1.515
MM	<code>detach</code>	0.005	0.004	0.004	0.007	0.007	0.009	0.019	1.000
MM	<code>push</code>	0.003	0.003	0.006	0.004	0.008	0.007	0.007	0.012

Three clean conclusions:

- **Pushforward is best or tied-best for every loss at every ϵ** , and it is also the cheapest (no rollout graph, so activation memory is a single forward). This is why it is the default in all later experiments.
- **Minimizing movement with full BPTT is catastrophic** ($0.41 \rightarrow 2.85$, i.e. worse than predicting nothing), while the same loss with the proximal centre detached is the best arm in the table. This is the quantitative version of §6.
- Detaching rescues least squares at large ϵ : `full` collapses from $\epsilon = 24$, `detach` survives to 28, `push` holds throughout.

8.4 `long_rollout` — extrapolation past the training horizon

Rolls the best `compare_bptt` checkpoints 20 steps (trained on 10) and tracks both the relative L^2 and the Ginzburg–Landau energy (5) against the CC reference, with divergence detection (non-finite frames are marked and the curve truncated). The energy check is the point: by (3) a rising E is a blow-up even if the error looks acceptable. Two-stage by design — the compute step runs on Euler where the checkpoints live and emits a small `metrics.json`; only that and the figures are transferred.

8.5 realistic_ac — the stiff, resolved regime with preconditioning

The current headline experiment: 128^2 , $\epsilon = 32$, $\Delta t = 0.01$, $K = 4$, 10-step pushforward rollout, 500 epochs, trained then rolled out to **100 steps**. Four arms: data-driven, minimizing movement, bare least squares, and **preconditioned least squares** with the multigrid V-cycle on $a^2 A + cM$. Relative L^2 against the CC reference on the test set:

loss	best space-time	step 10	step 50	step 100
data-driven	0.019	0.019	0.041	0.041
minimizing movement	0.008	0.008	0.014	0.013
preconditioned LS	0.047	0.034	0.039	0.037
bare LS	1.000	— (collapsed; excluded from the rollout figure)		

This is the cleanest statement of the Allen–Cahn result:

- **Bare least squares fails completely** — $\approx 100\%$ relative error. It does *not* produce NaNs; it stays finite and collapses towards the trivial prediction (final training loss 0.069, versus 3.8×10^{-3} preconditioned).
- **Preconditioning rescues it** to within a factor of two of data-driven training, using the same V-cycle machinery as Poisson with only the operator changed.
- **Minimizing movement is the best arm overall**, beating even data-driven training, consistent with it inheriting κ rather than κ^2 .
- **Rollouts stay stable to 100 steps** despite a 10-step training horizon; the error plateaus rather than growing, for all three working losses.

8.6 realistic_ac_K8 / realistic_ac_K16 — harder initial data

Identical protocol, with the multi-frequency initial condition at $K = 8$ and $K = 16$ instead of $K = 4$; separate output trees, analysis scripts reused unchanged. $K = 8$ is the interesting intermediate difficulty; at $K = 16$ **none of the arms converged**, which bounds how rich the initial data can be before the whole approach stops working at this resolution and budget. Locally only three of the four $K = 8$ runs are present and $K = 16$ results have not been pulled — worth completing before any of this goes in the paper.

8.7 bare_ls_seeds — is the bare-LS failure deterministic?

Three seeds of the bare (unpreconditioned) least-squares arm with the NaN guard removed, each writing to its own output directory, to distinguish “collapses to $\approx 100\%$ but stays finite” from “diverges to NaN”. This is the diagnostic that makes the failure claim in §8.5 safe to state.

9 Open issues and things worth checking

An unresolved inconsistency between two sweeps. `ac_stresstest` and `compare_bptt` have *byte-identical* CLI configurations apart from `--bptt_mode`, which `ac_stresstest` leaves unset — and unset resolves to `full_bptt` for the data and Galerkin losses. So `ac_stresstest`’s “data” and “LS” columns should reproduce `compare_bptt`’s full rows. They do not:

$\epsilon = 16$, data-driven	final train loss	best space-time rel. L^2	run date
<code>ac_stresstest</code>	1.5×10^{-1}	0.999	10 Jul
<code>compare_bptt</code>	1.0×10^{-5}	0.009	15 Jul

A *supervised* run failing to fit is not a plausible scientific finding, so the `ac_stresstest` large- ϵ collapse is most likely a stale-code artefact: the two sweeps are five days apart, their result JSONs differ five-fold in size (schema change), and the proximal-centre fix `ee1fdd0` landed the same day as the stress test. **Treat `compare_bptt` as authoritative and re-run `ac_stresstest` before using its numbers anywhere.** Note that this undercuts the “data-driven falls apart as ϵ grows” claim in the `ac_stresstest` README — `compare_bptt` shows data-driven holding to $\epsilon = 32$.

Stale READMEs. `experiments/allen_cahn/realistic_ac/README.md` describes 256^2 and $\epsilon = 64$ throughout, but `sweep.sbatch` actually runs 128^2 and $\epsilon = 32$ (the numbers reported in §8.5). The `ac_stresstest` README lists six ϵ values; eight were run.

The preconditioned-LS gap. Preconditioned LS (0.047) is still $6\times$ worse than minimizing movement (0.008) and $2.5\times$ worse than data-driven. The frozen-coefficient approximation $\mathbf{u}^2 \equiv 1$ is exact in the bulk and wrong exactly on the interface, which is where all the dynamics are — so this gap is expected, but it has not been measured against a non-frozen (per-step, per-sample) preconditioner. That is the obvious next experiment, and it is the one that would let us claim preconditioning closes the gap for Allen–Cahn as cleanly as it does for Poisson.

Not yet done. No mesh-refinement study for Allen–Cahn (the h^{-4} -versus- h^{-2} story is Poisson-only so far); no infinite-data-limit run (streaming datasets exist for Poisson only); no seed replication for anything except bare LS; and the ϵ -sweep has never been run with the preconditioned loss included, so we cannot yet say at which ϵ preconditioning stops rescuing least squares.